

Fiche d'investigation de fonctionnalité

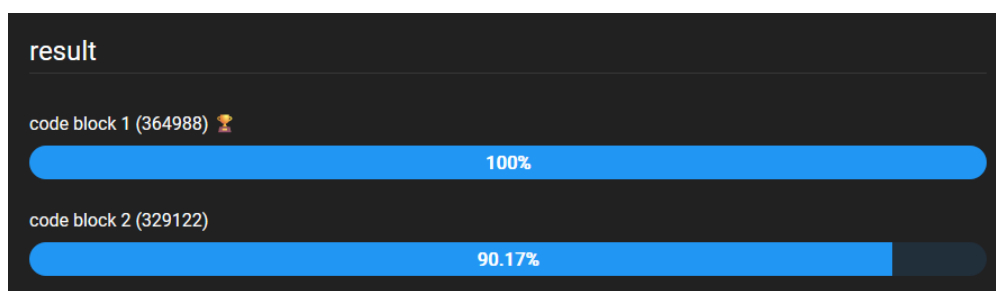
Fonctionnalité : Algorithme de recherche	Fonctionnalité #1
Problématique : L'utilisateur doit pouvoir rechercher la recette de son choix sur la page des recettes. Il doit pouvoir filtrer via des tags mais aussi via la barre de recherche principale.	

Option 1 : Boucles natives Cette méthode, appelée impérative, se base sur l'utilisation des boucles for et while lors de l'implémentation de l'algorithme. On parcourt les recettes une par une en utilisant un compteur. Pour filtrer, on a besoin d'un nouveau tableau pour y mettre les résultats. Le code vérifie si la recette actuelle correspond à la recherche. Si c'est le cas, la recette est ajoutée au nouveau tableau. Cette approche donne un contrôle total sur le déroulement du processus.	
Avantages ⊕ Contrôle total, permet d'interrompre ou sauter des itérations grâce à break et continue ⊕ Compatible sur tous les environnements Javascript	Inconvénients ⊖ Moins lisible, code plus verbeux ⊖ Risque d'erreurs avec la gestion de l'index et de la condition d'arrêt

Option 2 : Objet Array Cette méthode, appelée déclarative, se base sur l'utilisation des méthodes de l'objet Array en Javascript (foreach, filter, map, reduce). On filtre un tableau selon une condition et un autre tableau est renvoyé automatiquement. Cela permet de filtrer les recettes en retirant la gestion des compteurs et du tableau des résultats. Cette approche est plus moderne et rend le code plus simple à lire.	
Avantages ⊕ Bonne lisibilité, code plus concis et expressif ⊕ Immuabilité, renvoie un nouveau tableau sans risquer de modifier l'original ⊕ Pas de gestion d'index à faire manuellement	Inconvénients ⊖ Appel de fonction plus coûteux au niveau mémoire ⊖ Contrôle limité, impossible d'utiliser break ou continue pour arrêter l'itération. ⊖ S'applique uniquement aux tableaux

Solution retenue : Nous avons retenu l'approche qui utilise les boucles natives (for, while) car elles offrent un contrôle plus précis sur l'itération. Après différents tests, il s'avère que cette approche donne de meilleures performances lors de la recherche dans les recettes. Ce choix permet d'optimiser l'efficacité du code et de garantir une compatibilité maximale avec tous les environnements Javascript.
--

Voici la comparaison des deux algorithmes lors de la recherche principale de l'application.
En haut l'algorithme natif et en bas celui utilisant l'objet Array.



Annexes

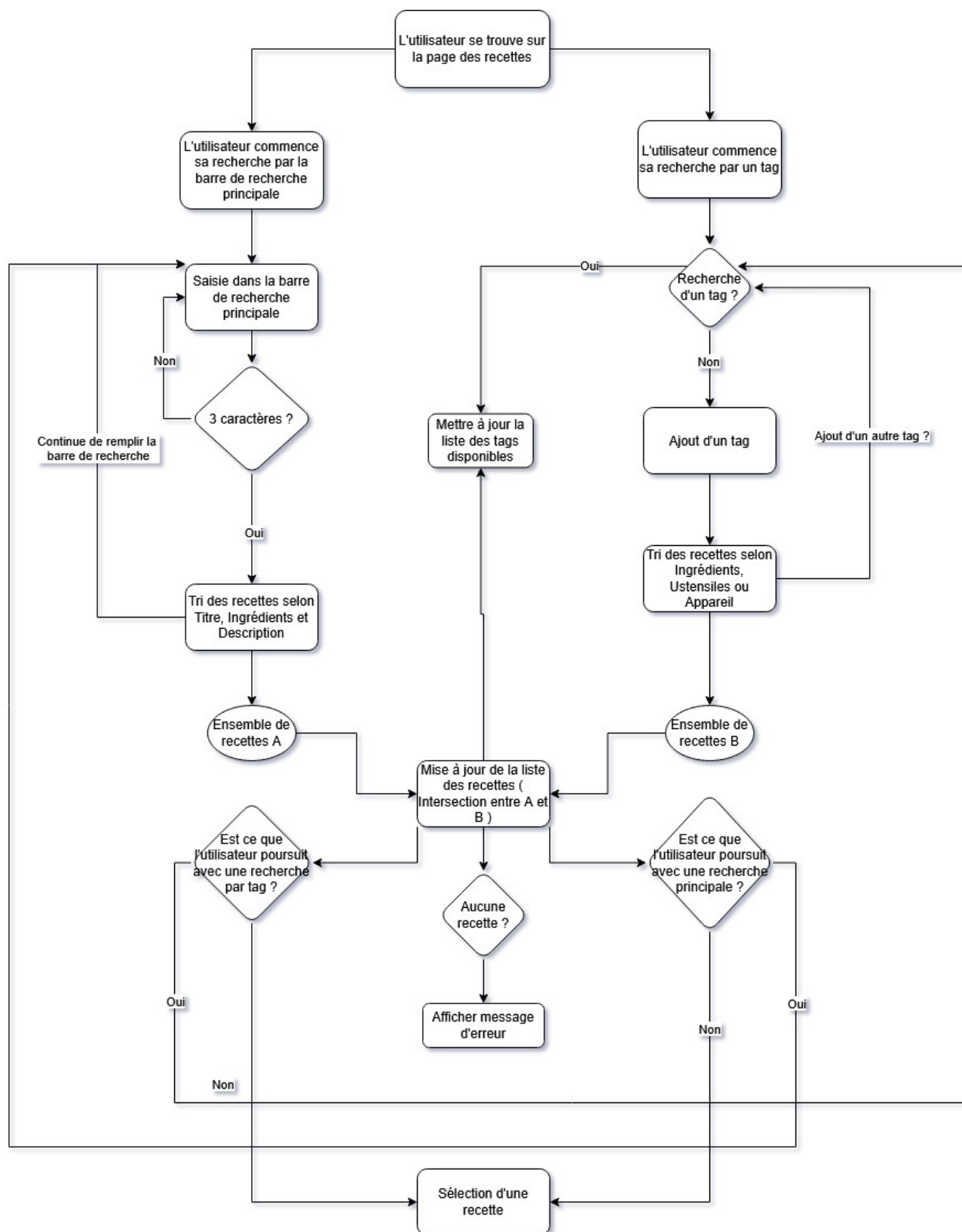


Figure 1 - Diagramme de l'algorithme de recherche